

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 - FLOWCHART-TO-CODE CONVERSION

Course Code: CSA0309 Course Title: Data Structures
CO Assessed: CO5 - Naturalization (BL4, BL6) Assessment: Assessment Tool 2 - Flowchart-To-Code Conversion
Weightage: 35% of CIA Total Marks: 100
CO5 Statement: Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.
Student Name: K. Navya Reg. No.: 192511231
Section / Batch: 19D4241D3 2nd year, SEM Date: 02/09/2026

Code of Conduct

I K. Navya (192511231) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Navya K. Navya

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Minimum Spanning Tree	Kruskal's Algorithm	2	20
Graph Sorting	Topological Sort	3	20
Graph Traversal	DFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

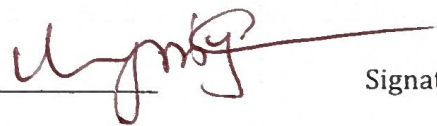
Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge

Course Coordinator

Program Director

Signature: 

Signature: _____

Signature: _____

Date: _____

Date: _____

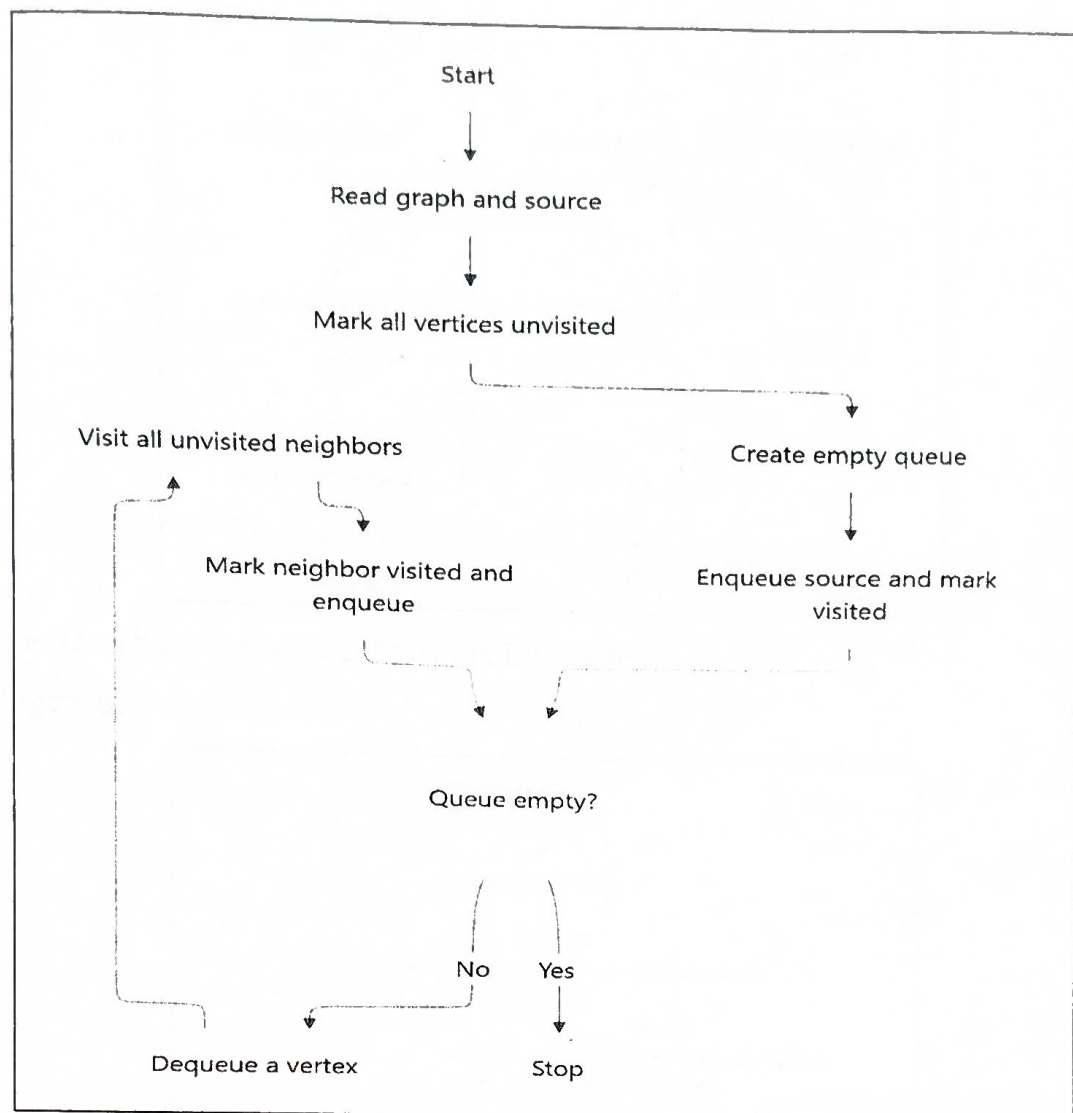
Date: _____

Annexure A – Flowchart-To-Code Conversion

1. Convert the flowchart for Breadth First Search traversal into code.

(20 Marks)

Flowchart Diagram:



2. Convert the flowchart for Kruskal's algorithm into code.

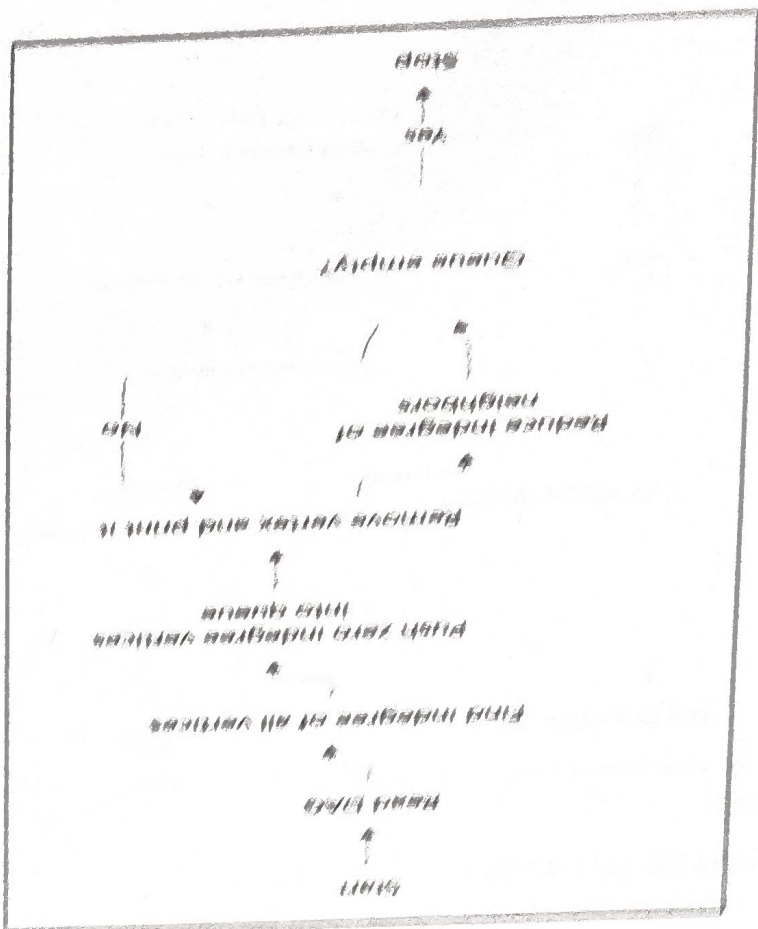
(20 Marks)

Flowchart Diagram:

Flowchart Diagram:

4. Convert the flowchart for generating a DFS tree into code.

(20 Marks)

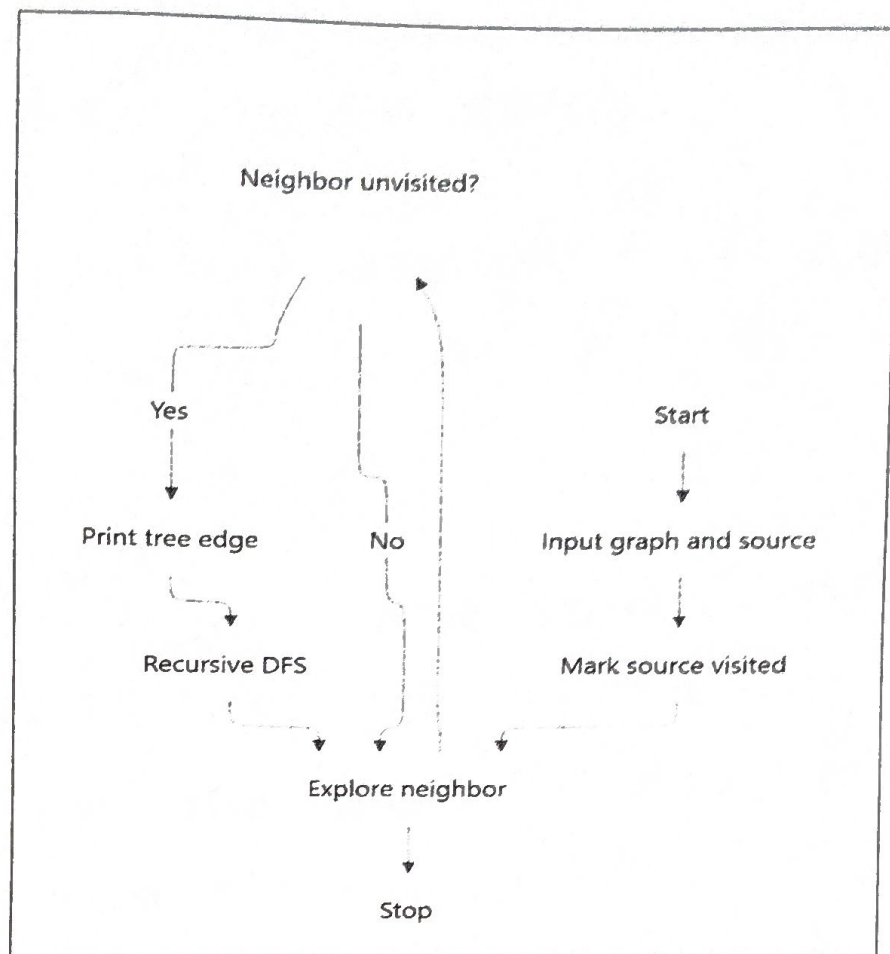


Flowchart Diagram:

3. Convert the flowchart for topological sort into code.

(20 Marks)

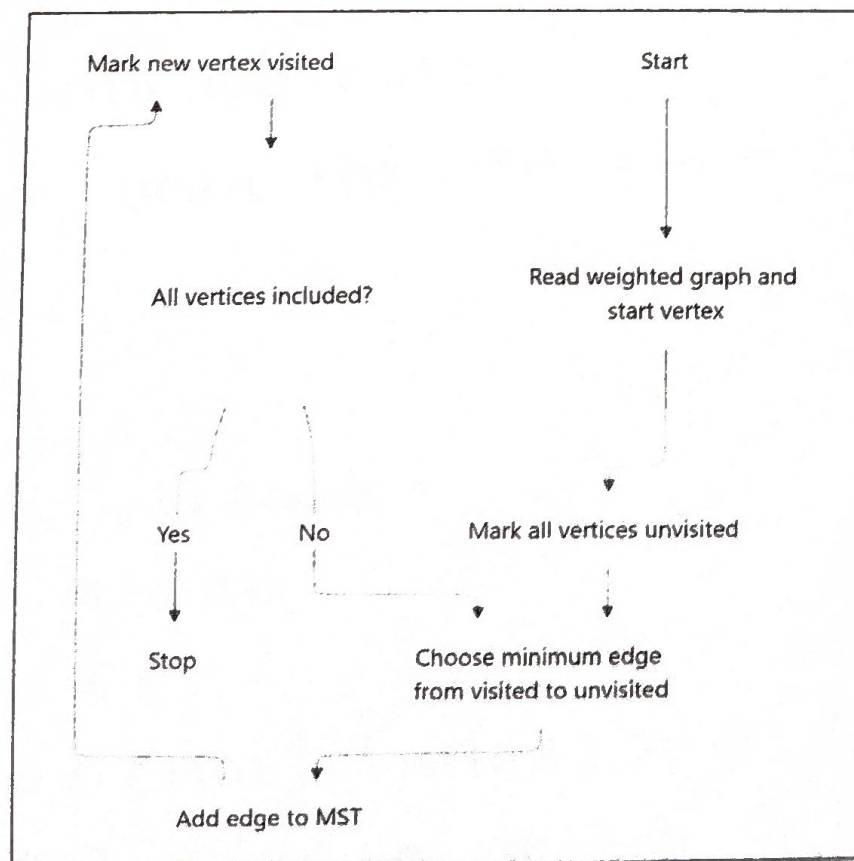




5. Convert the flowchart for Prim's algorithm into code.

(20 Marks)

Flowchart Diagram:



Breadth First Search (BFS):

Aim: To perform Breadth first search traversal of a graph using a queue.

Algorithm:

- (1) Read the graph and source vertex
- (2) Mark all vertices as visited
- (3) Insert all the source vertex into the queue and mark it visited.
- (4) Delete a vertex from the queue and print
- (5) Visit all its unvisited neighbours and insert them into the queue
- (6) Repeat until the the queue becomes empty.

C program:

```
#include <stdio.h>
#define MAX 20
int main () {
    int a [MAX][MAX], visited [MAX] = {0};
    int a [MAX], front = 0, rear = 0;
    int n, e, u, v, s, i, x;
```



```
printf ("Enter vertices and edges:");
```

```
scanf ("%d %d", n, &e);
```

```
for (i=0; i<e; i++) {
```

```
scanf ("%d %d", &u, &v);
```

```
    a[u][v] = a[v][u] = 1;
```

```
}
```

```
printf ("Enter source:");
```

```
scanf ("%d", &s);
```

```
q[rear++] = s;
```

```
visited[s] = 1;
```

```
printf ("BFS:");
```

```
while (front < rear) {
```

```
    x = q[front++];
```

```
    printf ("%d", x);
```

```
    for (i=0; i<n; i++) {
```

```
        if (a[x][i] & 1 & !visited[i]) {
```

```
            visited[i] = 1;
```

```
            q[rear++] = i;
```

```
        }
```

```
    }
```

```
}
```

```
return 0;
```

```
}
```

```
while [P[x] != x]
```

```
    x = P[x];
```

```
return x;
```

```
}
```

```
int main() {
```

```
    struct e e[20]t;
```

```
    int n,m,i,j,a,b;
```

```
    int count=0, cost=0;
```

```
    scanf ("%d %d", &n, &m);
```

```
    for (i=0; i<m; i++)
```

```
        scanf ("%d %d %d", &e[i].u, &e[i].v, &e[i].w);
```

```
    for (i=0; i<m-1; i++)
```

```
        for (j=i+1; j<m; j++)
```

```
            if [e[i].w > e[j].w]
```

```
            {
```

```
                t = e[i];
```

```
                e[i] = e[j];
```

```
                e[j] = t;
```

```
            }
```

```
printf ("MST:\n");
```

```
for (i=0; i<m & count < n-1; i++) {
```

```
    a = find [e[i].u];
```

```
    b = find [e[i].v];
```

```
    if (a != b) {
```

```
        printf ("%d - %d = %d\n", e[i].u, e[i].v, e[i].w);
```

```
        p[a] = b;
```

```
        cost += e[i].w;
```

```
        count ++;
```

sample Input:

4 5

0 1 10

0 2 6

0 3 5

1 3 15

2 3 4

Output:

MST

2 - 3 = 4

0 - 3 = 5

0 - 1 = 10

cost = 19

Sample output & Input:

5
1
2
3
4
5
0

BFS: 0 1 2 3 4 5

Kruskal's Algorithm:

Algorithm: To find the maximum spanning tree of a weighted graph using Kruskal's Algorithm

Algorithm:

- 1) Read the weighted graph
- 2) Sort all edges in increasing order of weight.
- 3) Initially put every vertex in a separate set
- 4) select the smallest edges.
- 5) If it does not form a cycle, add it to the MST
- 6) Repeat until $V-1$ edges are selected.

Program:

```
#include <std
```

```
struct e {
```

```
int
```

```
};
```

```
int P[
```

```
int t
```

print ("Cost = %d", cost);

return 0;

g.

Topological Sort:

Aim: To find the topological ordering of a directed acyclic graph using Khan's algorithm

Algorithm:

- (1) Read the directed graph
- (2) Find the in degree of every vertex.
- (3) Insert all vertices with in degree 0 into the queue.
- (4) Decrease the indegree of its neighbours
- (5) continue until the queue is empty.

(3) Topological Sort:

Aim: To find the topological ordering of directed acyclic graph using Khan's Algorithm.

Algorithm:

- (1) Read the directed graph.
- (2) Find the indegree of every vertex.
- (3) Insert all vertices with in degree 0 the queue
- (4) Decrease the indegree of its neighbours
- (5) continue until the queue is empty.

Code:

```
# include <stdio.h>
```

```
int main () {
```

```
int a[10][10] = {0}, in[10] = {0}, q[10],
```

```
int n, e, i, x, y, f = 0, r = 0;
```

```
scanf ("%d %d", &n, &e);
```

```
for (i = 0; i < e; i++) {
```

```
scanf ("%d %d", &x, &y);
```

```
    a[x][y] = 1;
```

```
    in[y] ++;
```

```
}
```

```
for (i = 0; i < n; i++)
```

```
    if (in[i] == 0)
```

```
        q[r++] = i;
```

```
while (f < e)
```

```
{ x = q[f++];
```

```
  print ("%d", x);
```

```
  for (i = 0; i < n; i++)
```

```
      if (a[x][i] && in[i] == 0)
```

```
          a[r++] = i;
```

```
}
```

```
return 0;
```

```
}
```


DFS Tree;

Aim: To generate a Depth first Search tree using recursive DFS.

Algorithm:

- (1) Read the graph and source vertex
- (2) Mark the source as visited.
- (3) Explore it's neighbour one by one.
- (4) Recursively perform DFS on that neighbour.
- (5) continue until all reachable vertices visited

C-code:

```
#include <stdio.h>
int a[10][10], v[10], n;
void DFS (int x) {
    int i;
    v[x] = 1;
    printf ("%d", x);
    for (i = 0; i < n; i++) {
        if (a[x][i] && !v[i])
            DFS(i);
    }
}

int main () {
    int e, x, y, s, i;
    scanf ("%d %d", &n, &e);
    for (i = 0; i < e; i++) {
```

$a[x][y] = a[y][x] = w;$

```

{
    v[0] = 1;
    for (i = 1; i < n; i++) {
        min = 999;
        for (x = 0; x < n; x++)
            if (v[x])
                for (y = 0; y < n; y++)
                    if (v[y] && a[x][y] && a[x][y] < min) {
                        min = a[x][y];
                        u = x;
                        z = y;
                    }
        print f ("%d - %d = %d\n", u, z, min);
        cost += min;
        v[z] = 1;
    }
    print f ("cost = %d", cost);
    return 0;
}

```

Sample Input:

5	7
0	1 2
0	3 6
1	2 3
1	3 8
1	4 5
2	4 7
3	4 9

Output:

0 - 1 = 2
1 - 2 = 3
1 - 4 = 5
0 - 3 = 6
cost = 16